

WORKSPACE AUTOMATION

# Automate your Calendar with Code

Learn how to extract Google Calendar events directly into Google Sheets using a simple Apps Script automation.

ABHILASH JOSE

THE FRAMEWORK

# Why Use Apps Script?

Google Apps Script is a cloud-based JavaScript platform that lets you integrate and automate tasks across Google Workspace products.

THE USE CASE: CALENDAR AUDITS

- **Freelancers:** Automatically generate timesheets for billing.
- **Managers:** Analyze how much time is spent in meetings.
- **Deep Work:** Audit your week to see where your hours actually go.

THE GOAL

We will build a script that asks the user for a timeframe, connects to Google Calendar, extracts event data, calculates duration, and writes it cleanly into a spreadsheet.

Automation Learn

File Edit View Insert Format Data Tools Extensions Help

My Audit Tools

Audit Calendar (Export Events)

A1

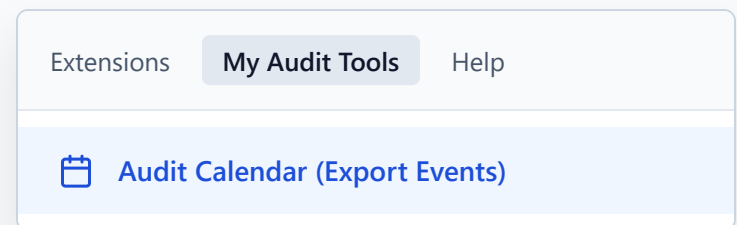
fx

Event Title

|   | A                         | B               | C               | D              | E        | F           |
|---|---------------------------|-----------------|-----------------|----------------|----------|-------------|
| 1 | Event Title               | Start Time      | End Time        | Duration (Hrs) | All Day? | Location    |
| 2 | Q2 Strategy Planning      | 3/19/2026 9:00  | 3/19/2026 11:00 | 2              | FALSE    | Google Meet |
| 3 | Deep Work: Report Writing | 3/20/2026 13:00 | 3/20/2026 16:00 | 3              | FALSE    | Focus Zone  |

# Creating Custom Menus

The `onOpen()` function is a special trigger in Apps Script. It runs automatically when the spreadsheet is opened, allowing us to build custom UI elements so users can run our script without opening the code editor.



*# Create a Custom Menu in Sheets*

```
function onOpen() {  
  
    var ui = SpreadsheetApp.getUi();  
  
    ui.createMenu('My Audit Tools')  
        .addItem('Audit Calendar (Export Events)',  
        'promptForCalendar')  
        .addToUi();  
}
```

# Getting User Input

We use a prompt box to ask the user how many days of calendar data they want to export. We also add logic to validate that they typed a real number.

```
# Ask user for Calendar Days
function promptForCalendar() {

    var ui = SpreadsheetApp.getUi();

    var response = ui.prompt(
        'Calendar Audit',
        'How many days into the future?',
        ui.ButtonSet.OK_CANCEL
    );

    if (response.getSelectedButton() == ui.Button.OK) {
        var days = parseInt(response.getResponseText());

        if (isNaN(days) || days < 1) {
            ui.alert('Please enter a valid number.');
        else {
            runCalendarAudit(days);
        }
    }
}
```

# Sheet Preparation

Before writing new data, we clear the active sheet to avoid overlapping old data, then we construct our bold header row.

```
# Setup the Sheet & Headers
function runCalendarAudit(days) {

    var sheet = SpreadsheetApp.getActiveSheet();
    sheet.clear();

    var headers = [
        "Event Title", "Start Time", "End Time",
        "Duration (Hrs)", "All Day?", "Location", "Status"
    ];

    sheet.appendRow(headers);

    sheet.getRange(1, 1, 1, headers.length)
        .setFontWeight("bold");
}
```

# Fetching the Calendar

We calculate our start and end dates based on the user's input, and then call the built-in **CalendarApp** service to retrieve the events.

```
# Define Timeframe
```

```
var start = new Date();  
var end = new Date();  
end.setDate(end.getDate() + days);
```

```
# Fetch Calendar Events
```

```
var calendar = CalendarApp.getDefaultCalendar();  
var events = calendar.getEvents(start, end);
```

```
# Prepare empty array for rows
```

```
var eventList = [];
```

# Processing the Data

We loop through the events. We use a **try/catch** block for status (in case an external event blocks permissions) and calculate the duration in hours by converting milliseconds.

```
# Loop through events
for (var i = 0; i < events.length; i++) {
  var e = events[i];

  # Handle Status Safely
  var myStatus = "";
  try { myStatus = e.getMyStatus(); }
  catch(err) { myStatus = "Owner"; }

  # Calculate Duration (ms to hours)
  var duration = "";
  if (e.isAllDayEvent()) {
    duration = "24";
  } else {
    duration = ((e.getEndTime() - e.getStartTime()) /
      (3600000)).toFixed(2);
  }

  eventList.push([
    e.getTitle(), e.getStartTime(), e.getEndTime(),
    duration, e.isAllDayEvent(), e.getLocation(), myStatus
  ]);
}
```



# Batch Writing Data

**Crucial Tip:** We write the entire array to the sheet at once using `setValues()`. Writing row-by-row inside a loop is extremely slow because it makes constant calls to the server.

```
# Write everything in one fast operation
if (eventList.length > 0) {

    sheet.getRange(
        2, 1,
        eventList.length,
        eventList[0].length
    ).setValues(eventList);

    SpreadsheetApp.getUi()
        .alert('Success! Found ' + eventList.length + ' events.');
```

```
} else {
    SpreadsheetApp.getUi()
        .alert('No events found in ' + days + ' days.');
```

```
}
```

```
}
```



THE RESULT

# Final Automation Output

The script automatically builds this structured table, ready for pivot tables or charts.

| Event Title               | Date      | Duration | Location     | Status  |
|---------------------------|-----------|----------|--------------|---------|
| Q2 Strategy Planning      | 3/19/2026 | 2 hrs    | Google Meet  | OWNER   |
| Deep Work: Report Writing | 3/20/2026 | 3 hrs    | Focus Zone   | OWNER   |
| Gym Session               | 3/21/2026 | 1.5 hrs  | Local Gym    | OWNER   |
| Project Alpha Sync        | 3/23/2026 | 0.75 hrs | Zoom Link    | INVITED |
| Conference: Tech 2026     | 3/24/2026 | 24 hrs   | Conv. Center | OWNER   |
| ...                       | ...       | ...      | ...          | ...     |
| ...                       | ...       | ...      | ...          | ...     |

## CORE CONCEPTS LEARNED

✓ **Custom Menus:** Using `onOpen()` to bridge code and the Google Sheets UI.

✓ **Data Handling:** Converting native Date objects into calculable hours.

✓ **Google APIs:** Integrating directly with CalendarApp securely.

✓ **Batch Writing:** Using `setValues()` with 2D Arrays for high-speed execution.

## BUSINESS VALUE & USE CASES

- **Automated Timesheets:** Stop manually tracking hours. Run this script on Friday and generate invoices for clients instantly based on calendar events.
- **Meeting Overload Audit:** Managers can audit team calendars to see exactly what percentage of the week is consumed by "Syncs" versus "Deep Work."
- **Resource Planning:** Easily pivot this data in Google Sheets to see which projects (based on keywords in titles) are taking up the most organizational bandwidth.

AUTOMATION COMPLETE

# Code. Automate. Save Time.

Apps Script bridges the gap between manual entry and seamless productivity. Start scripting your workflows today.

**ABHILASH JOSE**

Data Science Specialist